



Context Engineering: Stateless by Default, Stateful by Design

Giorgio Galassi
Senior Angular Consultant, Freelancer



Google
Developer
Groups

You open your coding agent.

You worked on a feature yesterday. You closed the session.

Today you open it again.

It has no idea who you are.

Three ways context fails

1. **Session amnesia** — everything resets when the session closes
2. **Cross-project blindness** — decisions made here never inform the next feature
3. **Tool switch loss** — context built in one tool is invisible to another

The instinct: fix it with a better prompt

"Here's what we discussed. Here's what we decided. Please continue."

It works once, but it doesn't scale. Every session you're rebuilding context that should have been persisted.

Prompt engineering vs Context engineering

Prompt engineering

What you say to the agent.

Context engineering

What the agent already knows before you say anything.

The discipline of deciding:

- **what** your agents know
- **when** they know it
- **how** that knowledge survives session boundaries and tool switches

What is context?

Think of the model as a CPU.

The context window is RAM: everything the model can see and reason about right now.

It has a hard size limit, resets between sessions, and degrades in quality as it fills up — often well before hitting that limit.

What competes for that space

Layer	What it contains
System prompt	Agent identity, rules, control flow
Tool definitions	Schemas for every tool the agent can call
Tool results	Outputs from previous tool calls
Retrieved knowledge	Documents, search results, RAG chunks
Conversation history	Everything said so far, every turn
Memory	Facts and preferences from previous sessions
Agent state	Current plan, progress markers, scratchpad

Left unmanaged, each of these layers contributes to a known failure mode.

When context isn't managed

- **Context confusion** — too many tools or irrelevant definitions cause wrong tool calls
- **Context distraction** — accumulated history causes the model to echo noise instead of reason
- **Context poisoning** — an error *or hostile* content enters once, then keeps being re-read — every session after, once it reaches memory
- **Context clash** — contradictory information produces inconsistent behavior

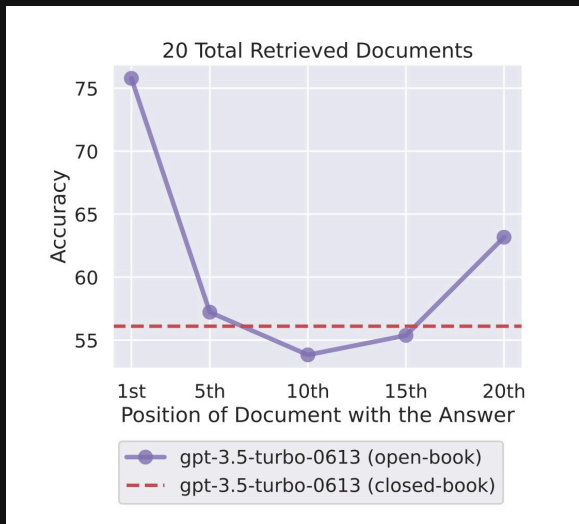
Three of these are hygiene. **Poisoning is a security problem** — because the output of agent A is the input of agent B.

Lost in the middle

Models don't degrade uniformly. They show a **U-shaped performance curve**: best recall at the start and end of context, worst in the middle.

Liu et al., **2023**: information buried in the middle dropped accuracy by **more than 20 percentage points** — below answering with no documents at all, on that year's models and tasks.

The curve has flattened since. It has not gone away.

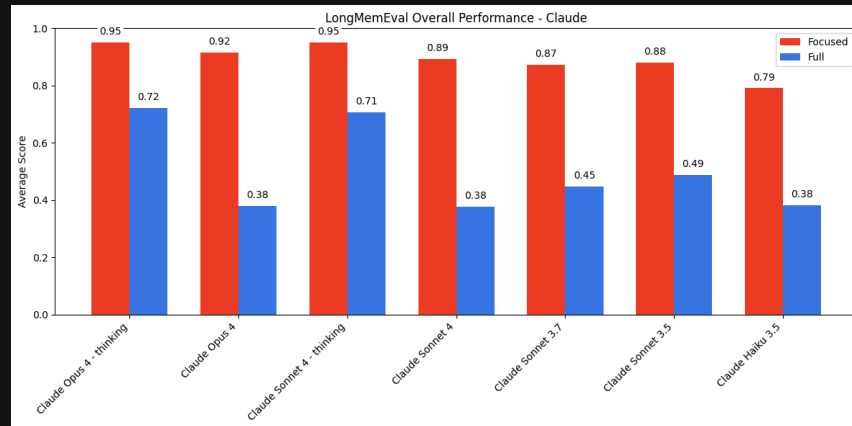
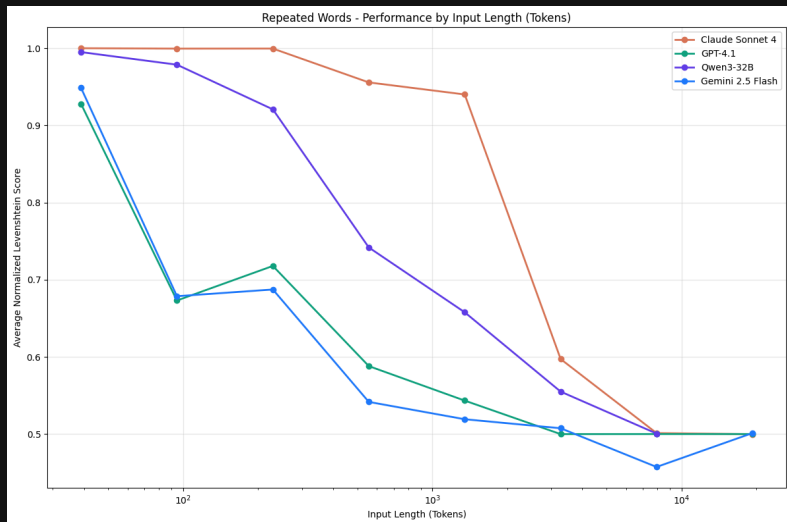


Context rot

Performance doesn't just degrade when context is full. It degrades as it fills — long before hitting any limit.

Chroma, July 2025 — 18 frontier models. **Every single one got worse** as input length increased.

Same question, same answer in context — **300 tokens** vs **113,000**.



Short-term vs long-term memory

	Short-term (working)	Long-term (persisted)
Scope	One session	Across sessions, tools, projects
Lives in	The context window	Files, database, memory service
Cost	Tokens on every single turn	Storage, plus a load when you ask
Fails by	Rot, distraction, overflow	Staleness, drift, contradiction
Cleared by	Closing the session	Only by you
Written by	The model, as it works	You, deliberately — or nobody

The context window is not memory. It's the desk. Long-term memory is the filing cabinet.

The long-term memory lifecycle

Four operations. Most setups implement two.

1. **Write** — what earns a place? Decisions, conventions, constraints. Not transcripts.
2. **Read** — retrieved by index, for a named phase. Not everything, not by default.
3. **Update** — two speeds. State is overwritten; decisions are marked `superseded` , never deleted. Drop the *why* and the rejected option comes back in three months.
4. **Forget** — expiry and demotion are features. Memory that only grows eventually stops being loadable.

Write and read make a memory system look like it works. **Update and forget are what make it trustworthy.**

The Techniques

1. Index-first loading

Prevents: session amnesia — the agent doesn't know where it is or what it's doing.

The problem: You reopen your session. The agent has no orientation. What project is this? What was the last decision? Where did you leave off?

The technique: Before reading anything else, load one small file that answers those questions. A dispatch table, not a document.

```
# AGENTS.md
```

```
Orient here first. This file points; it doesn't explain.
```

- Current state → `docs/state.md` (read first)
- Architecture → `docs/architecture.md`
- Specs → `docs/specs/` (one file per feature)
- Payment providers → `docs/notes/stripe-integration.md`
- Conventions → `docs/conventions.md`

This has a name now: `AGENTS.md` — stewarded by the Agentic AI Foundation under the Linux Foundation since December 2025, read natively by 20+ coding agents, used by **60,000+ open-source projects**.

2. Anchored iterative summarization

Prevents: context rot — accumulated noise degrading what the agent sees over time.

The problem: Yesterday's session produced a long conversation transcript. Transcripts are noisy, long, and expensive to load.

The technique: Summarize into a structured state document. **Overwrite it every session. Never append.**

Goal

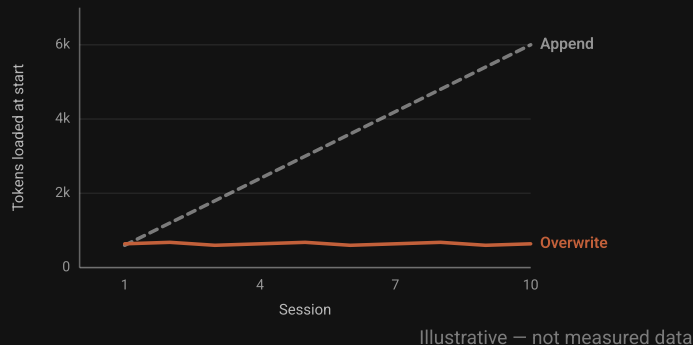
Redesign checkout flow to reduce drop-off at payment step.

Explored

- Option A: single-page checkout (rejected: too much refactor)
- Option B: progressive disclosure (selected)
- Option C: modal overlay (rejected: mobile UX concerns)

Open

- Payment provider: Stripe vs Braintree undecided



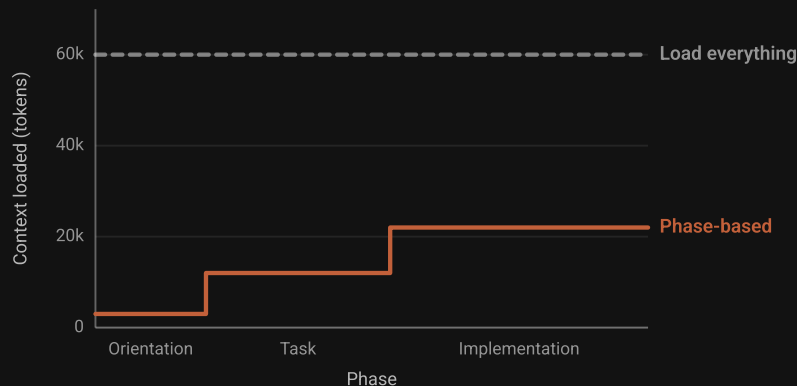
3. Phase-based, just-in-time loading

Prevents: context confusion — files and tools the current step never needs.

The problem: The agent doesn't need everything at once. Loading everything upfront wastes context window space.

The technique: Load in phases. Within a phase, load files — and tool definitions — the moment they're needed. Everything else stays on disk.

Phase	What loads
Orientation	Index file + state document
Task	Relevant specs + architecture notes
Implementation	Specific files being modified



Illustrative — not measured data

4. Sub-agent isolation

Prevents: context distraction — the search for the answer costing more than the answer.

The problem: To find one file, the agent reads twenty. Nineteen of them stay in the window for the rest of the session.

The technique: Run the dirty work — exploration, log triage, wide search — in a separate context. Only the conclusion comes back.



Illustrative — not measured data

```
Sub-agent reads:      20 files, ~40k tokens
Returns:             "Session handling: src/auth/session.ts:40, JWT + refresh"
Main context pays:   one line
```

Write · Select · Compress · **Isolate** — the four canonical context operations. Techniques 1–3 cover the first three.

The Decision Space

Where does memory live?

Three options. Each with a different tradeoff.

Location	Benefit	Cost
Inside the tool	Zero setup	Locked to one tool
Inside the repo	Versioned, reviewed, read by every tool	Only fits project knowledge
Outside both	Crosses projects, survives archiving	Requires discipline to maintain

Project memory goes in the repo. Memory about you goes outside it.

What storage primitive?

The instinct is to reach for infrastructure.

- Vector store — semantic search, embeddings, retrieval pipelines
- Database — structured queries, relationships, versioning
- MCP server — tool integration, manifest-based discovery

The question to ask first: *Do I have a search problem, or a retrieval problem?*

Who is allowed to write to memory?

Every memory entry is an instruction to a future session.

Nothing enters memory without a date, a source, and a review.

- **Date** — lets you expire a fact
- **Source** — lets you debug it
- **Review** — agent output never becomes agent belief without a human in between

How do you know your context is working?

You'll notice the symptoms before you notice any metric:

- The agent re-asks something you already answered
- It contradicts a decision from the same session
- It reaches for the wrong tool
- Answers get vaguer the longer the session runs

Three numbers worth watching:

- **Orientation cost** — tokens loaded before any work starts
- **Utilization** — how much of the loaded context the task actually touched
- **Useful turns** — how many turns before quality visibly drops

Test your context like you test your code

The golden set: ten to twenty questions that **only your memory layer can answer**.

```
Which payment provider did we choose, and what did we reject?  
What is the naming convention for API route handlers?  
Where did the last session stop, and why?
```

Ask them at the start of a session, from cold. If the agent can't answer from what it loaded, the memory layer failed — not the model.

- **Block on regression, not on score** — compare against the last known-good run
- **Ablate** — remove a file, halve the tools: does anything actually get worse?
- **Automate** — promptfoo or DeepEval can score the same set in CI on every change to your rule files

What this enables

For a single developer

The problem we started with: you reopen a session and the agent starts cold.

With context engineering:

- Resume a feature after three days with no re-explanation
- The agent knows the last decision, the open questions, the next step
- Switching models costs almost nothing: the context travels with you, not with the tool

At larger scale

The same discipline, applied further.

- A team sharing context across sessions and members
- Multiple tools reading the same memory layer: Claude Code, Cursor, a local model
- A shared write layer — for memory outside the repo, an MCP server every tool writes through

Four approaches, by what they treat memory as

Approach	Memory is...	Example	Lock-in
Managed memory API	Extracted facts, indexed for you	Mem0, Supermemory	The vendor's store
Temporal knowledge graph	Facts with a <i>valid from / until</i>	Zep / Graphiti	Graph schema + DB
Memory-first agent runtime	The agent's own editable state	Letta (ex-MemGPT)	The whole runtime
Filesystem-native	A file	AGENTS.md + markdown	None. It's a repo

Only one of these answers the staleness problem by design: **the temporal graph knows *when* a fact was true.**

The discipline is the same. The write layer is a choice.

What didn't change

Bigger windows. On-demand tools. Self-editing context. Managed memory. In a year, every **implementation** moved.

The questions underneath stayed exactly where they were:

- What does the agent need to know right now?
- What has to survive the session ending?
- What earns its seat in the default load?
- Who is allowed to write to what the agent believes?

The tools got a year better. **The decisions are still yours.**

Thank you

Stateless by default. **Stateful by design.**

Giorgio Galassi

Senior Frontend Engineer (Freelancer) · GDG Roma Città Organizer

Angular

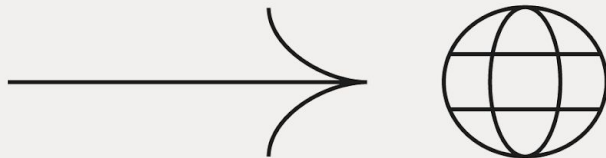
AI Workflows

Context Engineering



Links, slides & more





2026



Google
Developer
Groups